

Test effort estimation in early stages using Use Case and Class models for web applications

Pulak Sahoo* and J.R. Mohanty

School of Computer Engineering, KIIT University, Bhubaneswar 751024, India

Abstract. The testing phase of Software Development Life Cycle (SDLC) is considered essential for delivering the quality expected from the software product. The quality of the software product determines its acceptability and preserves the reputation of the developing organization. Early estimation of test effort is a significant predicament faced by the software managers. An early estimation enables early test planning which helps the project to meet the timeline, as well as budgetary and quality goals. Early estimation of the test effort is a relatively unexplored territory with limited undertaken research work. Unified Modeling Language (UML) is a widely accepted standard for representing Object Oriented Systems (OOS) during different phases of SDLC. Selected UML models created with adequate amount of information can be utilized for early test effort estimation. This work proposes a simple, semi-automated two-stage approach to estimate system test effort from Use Case and Class models during requirements and design phases by using their inherent attributes as parameters. As per the experiments conducted on a number of Web-based projects, the approach presented in this work predicted the test efforts with reasonable accuracy.

Keywords: Use Case model, Class model, UCPs, UCPM, CPs, TCF

1. Introduction

Quality of software products is a major concern for the customers and the developing organizations over the years. Inadequate quality brings unfavorable customer response causing damage to organizations' reputations. Hence, Software Testing is given utmost importance during SDLC, resulting in organizations spending up to 40% of the total development effort on testing [21]. For efficient and effective conduction of test activities, proper planning based on accurate estimation is essential. Objective of most traditional methods is to estimate the software development effort. The estimated development effort includes test effort which is a disadvantage, especially for IT organizations focusing on testing as predominant business [30]. An

early estimation of test effort gives distinct advantages to the project team. They can now plan ahead and allocate suitable resources to various test activities resulting in timely completion of the project with required quality.

Early Test effort estimation is a relatively uncharted territory and has many challenges [19,23]. During the initial stages, information available about the product is not sufficiently detailed [2]. Only with progression of time, further details about the product are discovered. This necessitates a multi-stage approach to test effort estimation. This work proposes an initial estimation obtained during the requirements stage followed by a second more refined estimation obtained during the design stage.

UML is the de facto standard for depicting software products developed using Object Oriented Methodology (OOM) [8]. UML prescribes a number of models to be constructed during different phases of SDLC. Experts over the years have confirmed that UML models constructed with required information can measure the

*Corresponding author: Pulak Sahoo, School of Computer Engineering, KIIT University, Campus-15, Bhubaneswar 751024, India. Tel.: +91 674 6550115; E-mail: saahoo_pulak@yahoo.com.

size and complexity of software products with reasonable accuracy [9,16,17,19,26,29]. Recent trends show that researchers are now using the inherent information stored in UML models for software testing purposes [27]. In addition to this, various studies have shown that UML models can be utilized for generating test cases with each UML model type playing a different role in testing [20,25,28]. It has also been established recently that UML models can be used for test effort estimation [7,23,24,30].

This work proposes a two-stage approach to estimate early system test effort from Use Case and Class UML models constructed during requirements and design phases. The selection of Use Case and Class models for test effort estimation is based on below reasons. Firstly, it is well known that Use Case models contain information from a customer's viewpoint while Class models contain information from a developer's viewpoint [15]. Hence, selecting these two models will cater to both the perspectives. Secondly, both Use Case and Class models are constructed well before the remaining UML models enabling managers to apply the estimation process early.

Use Case models are extremely popular and are the first type of UML models developed during requirements stage. They depict functional requirements and user interfaces of the system. The main elements of Use Case model are Use Cases, Actors and their Associations. Use Cases contain Use Case scenarios consisting of sequence of steps to achieve a specific functionality. This information is essential for System/Conformance testing used to decide whether a product meets its functional requirements or not [9]. Early availability of Use Case models makes them the most suitable candidates for initial system test effort estimation. This initial estimation enables managers to start early with test planning activities [23,24]. The estimation process will be undertaken again when more information about the system is available at a later stage.

Class models are the second type of UML models developed during the design stage depicting the classes and their relationships. Classes are the basic structural units of an OOS. The functionalities offered by the system are distributed among the classes making them ideal candidates for more accurate test effort estimation. Hence, Class models are utilized for system test effort estimation during the design stage. Remaining popular UML models like Sequence, Activity and Statechart models are constructed later in the development life cycle. The effort estimation process can then be refined further using these models [23].

The remainder of this paper is organized as follows. Section 2 (related work), describes a number of important studies conducted in the area of development and test effort estimations. Section 3 (proposed approach), explains the early system test effort estimation approach based on Use Case and Class models. Section 4 (experimental studies and results analysis), contains the details of the experiments conducted on a number of case studies and analyzes the results obtained. Section 5 (conclusions and future work), summarizes the findings of this work and discusses the scope of future work.

2. Related work

During last three decades, experts have proposed several innovative methods for estimating size, complexity and efforts for development and testing of software products. As part of the literature survey, a number of relevant estimation methods were studied and summarized below along with their pros and cons.

Function Point Analysis (FPA) is one of the earliest trendsetting methods proposed by Albrecht for estimating size of a software product [1]. FPA is a well established early stage implementation-independent method to estimate size of a product in terms functionalities offered. The size of the product is expressed in function points (FPs), which is then used for estimating the development effort by taking into account Technical Complexity Factors (TCF) and productivity factor (PF) of the organization. TCF is computed as per formula in Eq. (1):

$$TCF = C_1 + C_2 \sum_{i=1}^n F_i \quad (1)$$

Where $C_1 = 0.65$, $C_2 = 0.01$ and F_i are the TCFs measured in a scale of 0 to 5.

The FP size and development efforts are then computed using Eqs (2) and (3).

$$FP = UFP * TCF \quad (2)$$

$$Effort = FP/PF \quad (3)$$

Where UFP is unadjusted FP count and PF is the productivity factor.

In the year 1996, Caper Jones found a strong relationship between FP count and number of system test cases for testing a software product [13]. He proposed that the approximate number of system test cases can be computed from FP count using the formula in Eq. (4).

$$\text{Number of Test Cases} = (FP)^{1.2} \quad (4)$$

The COCOMO model is another widely acknowledged model for estimating the development effort from product size in SLOC (Source Lines of Code) [5]. The effort estimation uses a linear regression formula taking into account effort multipliers defined based on development processes and environment [31]. The estimation process is refined through three stages called Basic, Intermediate and Detailed COCOMO.

Test Point Analysis (TPA) is a well known test effort estimation method proposed by Van Veenendaal and Dekkers [6]. It is based on the results of FPA and is used to estimate the system test effort in form of Test Points (TPs). TPA method takes below parameters as inputs to produce test effort estimation. The TPA method also takes into account the environmental factors influencing testing activities.

- (1) Size of the product in FPs
- (2) Testing strategy (quality characteristics required to be tested and their relative user importance)
- (3) Productivity factor specific to the organization (number of test hours needed to realize a TP)

The FPA based methods mentioned above are implementation-independent functionality-based estimation methods that can be used in early stages of SDLC. Unfortunately, FPA have a number of shortcomings. They require collection of detailed requirements to produce accurate estimations which are not always available in the early stages. Apart from this, FPA based methods are suitable for products designed using Function Oriented Approach and unsuitable for present-day OOSs [30].

Karner introduced the well-accepted Use Case Points Methodology (UCPM) for early estimation of development effort from software's Use Cases which is a source of many ongoing innovations [10]. UCPM approach employs the Use Cases representing the functional complexity as inputs to produce development effort in terms of Use Case Points (UCP) [4,31]. It segregates information in each Use Case into four categories: Actors, number of transactions, Technical Factors and Environmental Factors. This information is then converted into points and summarized as Unadjusted Use Case points (UUCP). The UCP estimation is obtained by multiplying UUCP to project specific Technical Complexity Factor (TCF) and Environmental Factor (EF) (Eq. (5)). The final test effort is obtained by multiplying the UCP with Conversion Factor (CF) defining project team's productivity (Eq. (6)).

$$UCP = UUCP * TCF * EF \quad (5)$$

Where, *UUCP* is the Unadjusted *UCP*, *TCF* is the Technical Conversion Factor and *EF* is the Environmental Factor.

$$Effort = UCP * CF \quad (6)$$

Where, *CF* is the productivity of the project team.

Nageswaran proposed an innovative extension to UCPM approach to estimate acceptance testing effort [30]. Pre-requisites to his method are early availability of system level Use Cases and selection of V-model as the SDLC approach. V-Model advocates preparation of the acceptance test plans in the requirement phase. Given below are the estimation steps:

- (1) Identification and classification of Actors (based on type of interface (GUI, Protocol-Driven or API))
- (2) Computation of Unadjusted Actor Weight (UAW)
- (3) Identification and classification Use Cases (based on number of transactions/scenarios)
- (4) Computation of Unadjusted Use Case Weight (UUCW)
- (5) Computation of Unadjusted Use Case Points (UUCP) using Eq. (7)

$$UUCP = UAW + UUCW \quad (7)$$

- (6) Computation of Adjusted Use Case Points (AUCP) by considering 9 Technical and Environmental Factors (TEF) (Eq. (8))

$$AUCP = UUCP * [0.65 + (0.01 * TEF)] \quad (8)$$

- (7) Computation of test effort by multiplying the AUCP with CF defining productivity (Eq. (9))

$$Effort = AUCP * CF \quad (9)$$

CF is defined as man-hours required for planning, writing and conducting tests on one UCP. Nageswaran has considered a fixed CF value of 20 based on Enterprise Java Beans technology.

Nageswaran's approach is simple and suitable for early acceptance test effort estimation. It also has below limitations. Firstly, categorization of Actors and Use Cases as simple, average and complex seem inadequate for modern-day software products. Secondly, all Use Case transactions/scenarios are given same score irrespective of their type's (normal or exception). Finally, the CF defining productivity is considered to be independent of EFs, which may not be correct [11].

In 2009, de Almeida et al. suggested the UCPM based N-Weighted method, which is a refinement over Nageswaran's approach with one major enhancement [7]. It classifies the Use Case scenarios into two

types: normal and exception. This is based on the observation that normal scenarios have greater number of checks than exception scenarios that result in well defined error messages. Hence, more effort is required to test a normal scenario resulting in a higher weight assignment. The weights assigned to both types of scenarios are determined based on historical data. The Use Case weight (P_T) is derived using the below formula (Eq. (10)):

$$P_T = P_N * N + P_E * E \quad (10)$$

Where, P_N and P_E are the weights of the normal and exception scenarios, N and E are the No. of normal and exception scenarios present in the Use Case.

Rests of the effort estimation steps are similar to Nageswaran's approach. Two additional estimation approaches (NSM/MIS and Simplified Methods) aimed at simplifying UCPM are also discussed in this work. While these methods simplify the effort estimation process, the margin of error may be higher due less information being considered as input.

In 2011, Ochodek et al. applied cross-validation process to compare the accuracies of the different variants of UCPMs and proposed a simplified approach to estimate the development effort [19]. By performing factor analysis through subject matter experts, he observed that some Adjustment Factors have minor influence on estimation accuracy and some others are overlapping. He then reduced the number of Technical and Environmental Factors from 21 to 6 (4 TCFs: Efficiency, Operability, Maintainability, Interoperability and 2 EFs: Team experience, Team cohesion). His study suggested that the UCP computation based on count of Use Case steps were more accurate than counting transactions. Ochodek's approach simplifies UCPM approach without conceding estimation accuracy. As mentioned by the authors, the major limitation in this study is related to small size and lack of diversity of case study projects. Moreover, this approach is used to estimate system development effort not test effort.

Costagliola et al. presented the Class Point (CP) method in 2005, which estimates the size and development effort of OOSs using the information present in its Classes [9]. Since, Classes are the logical building blocks of an OOS containing information from a developer's viewpoint; the effort estimation using Classes is more likely to produce accurate results. In CP approach, two measures named CP1 and CP2 are proposed. CP1 is intended to provide a preliminary estimation in early stages, which can be refined later by applying CP2 when more information is available.

The CP estimation process has four steps as described below:

- (1) Analysis of the design documents to identify and then classify the Classes into four types (the problem domain, the human interaction, the data management, and the task management).
- (2) Allocation of complexity level to each Class based on the number of methods and its interaction with remainder of the system. This is done in two stages.
 - (a) CP1 – An initial size estimation computed based on weights assigned to the classes using two measures: NEM (Number of External Methods) and NSR (Number of Services Requested).
 - (b) CP2 – A refinement of CP1 based on an extra measure called NOA (Number of Attributes) in addition to NEM and NSR.
- (3) The Total Unadjusted Class Point (TUCP) is obtained by summing the Class weights.
- (4) The Class Point (CP) of the OOS is computed by multiplying TUCP with TCF. The TCF is determined from the Total Degree of Influence (TDI) of 18 general system characteristics (Eqs (11) and (12))

$$TCF = 0.55 + (0.01 * TDI) \quad (11)$$

$$CP = TUCP * TCF \quad (12)$$

Based on experiments conducted with 40 student projects, the aggregated MMRE of CP1 and CP2 were computed to be 0.19 and 0.18 [16]. Costagliola's CP approach is widely used to estimate the software development effort. More research is needed to explore the applicability CP Method for test effort estimation [3].

Satapathy et al. proposed an estimation approach which suggested a number of enhancements to CP Method [29]. The main improvement suggested is the use of soft computing techniques to optimize the effort parameters [14]. The other enhancements include: use of four complexity levels (low, average, high, and very high) to categorize classes instead of three and use of 24 General System Characteristics (GSC) instead of 18 to compute TCF. The proposed method was implemented using Multi-Layer Perceptions (MLP) and Radial Basis Function Network (RBFN). The authors have concluded that the MLP model gives higher estimation accuracy compared to the RBFN model.

In 2006, Kim et al. proposed an automated approach for development effort estimation in terms of UML points [16]. UML Points is calculated by combining both UCPs and CPs after applying refine-

ments. To calculate UCP and CPs, a number of additional metrics are taken into account. The metrics used for UCP calculation are: NOA, NOUC, NOR, ANA_UC and ANR_UC. The metrics used for CP calculation are: NOC, NOIR, NOUR, NORR, NOM, NOP, NOCA, NOASSOC, ANM_CLS, ANP_CLS, ANCA_CLS, ANASS_CLS and ANREL_CLS. Using these metrics, UCPs and CPs are computed based on the formulae in Eqs (13)–(15). Finally, UML points are computed by adding UCPs to CPs. UML Points approach uses 13 TCFs and 8 EFs for the effort adjustment.

$$UCP = \sum (NOA + NOUC + NOR) \quad (13)$$

$$CP = \sum (NOC + NOIR + NOUR + NORR + NOM + NOCA + NOASS) \quad (14)$$

$$UML\ Points = UCP + CP \quad (15)$$

UML Points approach addresses two major concerns in UCP and CP Methodologies. Firstly, the insufficiency of information used in calculation of UCPs and CPs. Secondly, over reliance on expert's decision regarding weights assigned to Use Case and Class model components. Instead, it focuses more on the models by using a number of newly defined metrics reducing dependency on expert's decision. However, further research is required to refine the UML Points approach for estimating test effort. More analysis is needed to understand the relationships between metrics used and complexity of the model components.

Sarma and Mall presented a novel approach to generate system test cases from Use Case and Sequence diagrams [20]. The steps include conversion of Use Case diagram to Use Case Diagram Graph (UDG) and Sequence diagram to Sequence Diagram Graph (SDG). Then, both UDG and SDG are combined to form the System Testing Graph (STG). The STG is then traversed in two levels to generate the test cases. This work can be extended for estimating system test effort. Once test cases are generated, the next step can classify them based on complexity and assign appropriate weights. Then, the test effort can be computed by summing up the weights and applying relevant TCF/EFs. Since, Sequence diagrams are only available later in the design stage; this approach will not be able to produce test efforts early enough.

Swain et al. proposed a technique to generate test cases using Statechart and Activity diagrams [28]. The steps involved are: (1) Creation of an intermediate model called the State-Activity Diagram (SAD) from

the State and Activity diagrams, (2) Generation of test cases by traversing all the basis paths in the SAD. In 2013, Anbunathan and Basu proposed a similar method to generate test cases from Class and Statechart diagrams [25]. Once test cases are generated, the test effort can be estimated in the same manner as described in paper [20]. Unavailability of Activity and Statechart diagrams in early stages of SDLC proves to be a limitation with these approaches. The other issue with these approaches is that, a great deal of effort is spent on generating the test cases before estimating the test effort [20,25,28].

Based on extensive study and analysis of several notable estimation methods including the ones specified in this section, below inference can be drawn. Despite of the obvious benefits exhibited by the currently available estimation methods, there exist many limitations such as: lack of an early estimations, inaccuracies in estimation, complexities of the estimation process, single-stage estimation processes, labor-intensive processes of gathering input metrics and over-reliance on expert decision instead of historical data.

Therefore, it is evident that there is a necessity for developing an uncomplicated, cost-effective, efficient, automated and reasonably accurate approach for early test effort estimation. This work offers a semi-automated two-stage approach for early system test effort estimation based on Use Case and Class models. Based on these estimations, the project team can start planning for the system testing activities early. This approach can be extended further by revising the earlier estimates in later stages when remaining UML models are available.

3. Proposed approach

The approach proposed in this work mainly focuses on contemporary Web-based projects. However, it can also be applied to any system developed using OOM represented by UML models. This work proposes a two stage estimation approach by applying Use Case models available in the requirements stage followed by applying Class models available in the design stage using their inherent attributes [23]. The steps of the proposed estimation approach are explained below.

3.1. Step-1: Test effort estimation using Use Case models

After finalizing the requirements, Use Case models are prepared by the project team to describe the

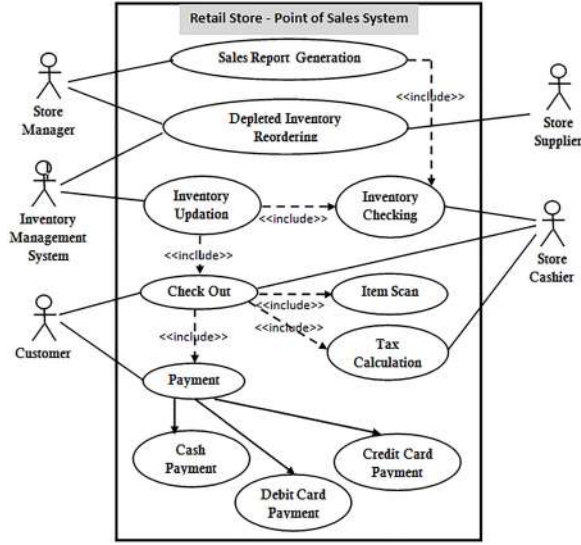


Fig. 1. Retail store point of sales system – top level Use Cases.

system functionality from a customer's viewpoint. The creators of UML standards recommended that Use Case models should be used as the basis for various types of testing including system and conformance testing [4,8,12,27]. Hence, Use Case models are suitable for early system test effort estimation. The main components of a Use Case model are: Use Cases, Actors and Interactions between them. Figure 1 shows an example Use Case model of "Retail Store Point of Sales" system along with its components [32].

3.1.1. Step 1.1: Use Cases weight determination

As per Jacobson, a Use Case contains the complete sequence of steps set off by an Actor while interacting with a functional unit of the system [12]. The System-level Use Cases used in this work; contain Use Case scenarios consisting of steps written in natural language [3,19]. The Use Case scenarios can be classified into two categories: main and exception scenarios. The main scenario comprises of the sequence of steps necessary achieve a functional objective. The exception scenario includes the steps specific to error handling. In this work, the scenario steps are considered as the basic element of a Use Case.

From numerous studies, it has been observed that, testing of the main scenarios require greater number of checks as opposed to the exception scenarios [7,19]. Hence, a higher weight ($W_M = 1$) is assigned to steps in main scenarios compared to steps in exception scenarios ($W_E = 0.4$). The values of W_M and W_E are determined based on the studies conducted in paper [7]

and validated through the experiments conducted in this work. Furthermore, the analysis done in this work shows that the 'number of Actors interacting with a Use Case becomes a multiplying factor to determine the total Use Case weight. This is because a Use Case must be tested from each interacting Actor's perspective.

Based on studies conducted in this work, it is concluded that the testing complexity of a Use Case is dependent on two factors: (1) the number of main and exception scenario steps present in the Use Case and (2) the number of Actor's interacting with it. Hence, the proposed method expresses the Use Case weight as a function of these two inherent parameters (Eq. (16)).

$$UseCase_{weight} = f_{ucw}(\#Scenario_Steps_{main}, \#Scenario_Steps_{except}, \#Interacting_Actors) \quad (16)$$

Where, $\#Scenario_Steps_{main}$ and $\#Scenario_Steps_{except}$ are the number of steps present in main and exception scenarios, $\#Interacting_Actors$ is the number of interacting Actors.

The Total Use Case weights are calculated based on the formula given in Eq. (17). Table 1 contains four Use Cases (taken from one of our case studies) with corresponding weights calculated using the proposed approach.

$$UseCase_{weight} = (W_M * \#Scenario_Steps_{main} + W_E * \#Scenario_Steps_{except}) * \#Interacting_Actors \quad (17)$$

Where, $W_M = 1$ and $W_E = 0.4$ are the weights of the main and exception scenario steps.

3.1.2. Step 1.2: Actor weight determination

Actors (human users or interacting systems) are integral parts of a Use Case model. They interact with Use Cases through a sequence of transactions. Studies done by many notable works have established that an Actor's complexity from the testing perspective depends on the type of communication with Use Cases. The four different communication types are: (1) GUI, (2) API, (3) Protocol-Driven Interface and (4) Data Store. Based on the analysis done in this work, it is concluded that the testing complexity of an Actor depends on the type of communication with the Use Cases. Hence, the proposed method expresses the Actor weight as a function of the communication type (Eq. (18)).

$$Actor_{weight} = f_{acw}(Communication_Type) \quad (18)$$

Table 1
Weight calculation for Use Cases

Use Case name	No. of steps in main scenarios (m)	No. of steps in exception scenarios (e)	No. of Interacting actors	Total Use Case weight (formula in Eq. (17))
View crime report	3	1	2	6.8
Fill FIR form	6	3	1	7.2
View case status	3	1	2	6.8
Give feedback	5	2	1	5.8

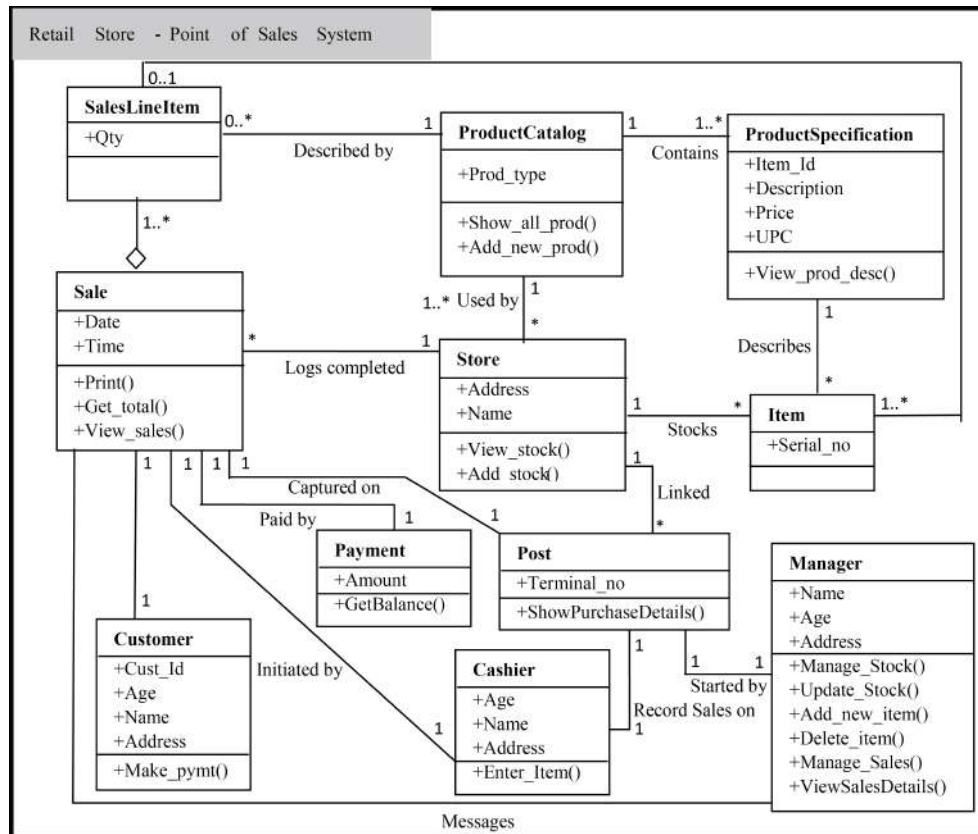


Fig. 2. Retail Store Point of Sales system – Class model.

Table 2
Actor complexity categorization

Actor class	Description of interaction with Use Case	Weight
Simple	End-user interacting through GUI	1
Medium	Person interacting through a Protocol-Driven Interface	2
Complex	Person/External system interacting through API	3
Very complex	External system interacting through a common Data Store	4

Where, *Communication_Type* is the type of communication between the Actor and the Use Case.

During literature survey, it was found that most of the studies have used only three complexity categories (simple, average/medium and complex) to categorize

the Actors. From the experiments conducted in this work, it was derived that using four categories (simple, medium, complex and very complex) covers a wider range of projects yielding better estimation accuracy. Table 2 contains the Actor complexity classes and corresponding weights proposed in this approach. The weights assigned to the classes are well established in previously conducted research works and verified through our experiments [7,19,30].

3.1.3. Step 1.3: Weight determination for Use Case models

The system under estimation may contain a number of Use Case models. The weight for each Use

Table 3
Class complexity categorization

0–2 #Services requested	0–5 #Attributes	6–9 #Attributes	10–14 #Attributes	> 10–14 #Attributes
0–4 #Methods	Simple	Simple	Medium	Complex
5–8 #Methods	Simple	Medium	Complex	Complex
9–12 #Methods	Medium	Complex	Complex	Very_Complex
> 12 #Methods	Complex	Complex	Very_Complex	Very_Complex
3–4 #Services requested	0–4 #Attributes	5–8 #Attributes	9–13 #Attributes	> 13 #Attributes
0–3 #Methods	Simple	Simple	Medium	Complex
4–7 #Methods	Simple	Medium	Complex	Complex
8–11 #Methods	Medium	Complex	Complex	Very_Complex
> 11 #Methods	Complex	Complex	Very_Complex	Very_Complex
> 4 #Services requested	0–3 #Attributes	4–7 #Attributes	8–12 #Attributes	> 12 #Attributes
0–2 #Methods	Simple	Simple	Medium	Complex
3–6 #Methods	Simple	Medium	Complex	Complex
7–10 #Methods	Medium	Complex	Complex	Very_Complex
> 10 #Methods	Complex	Complex	Very_Complex	Very_Complex

Case model is determined by combining the Use Case weights and Actor weights. Finally, the Total weight of the system is obtained by summing the Use Case model weights (Eq. (19)).

$$UseCaseModel_{weights} = \sum_{ucm_1}^{ucm_n} (UseCase_{weight}) + \sum_{ucd_1}^{ucd_n} (Actor_{weight}) \quad (19)$$

Where, $UseCaseModel_{weights}$ is the Total Weight of the system consisting of n Use Case models (ucm_1 to ucm_n), $\sum_{ucm_1}^{ucm_n} (UseCase_{weight})$ and $\sum_{ucd_1}^{ucd_n} (Actor_{weight})$ are the sum of Use Case and Actor Weights.

3.2. Step 2: Test effort estimation using Class models

Class models containing structural elements of the system are developed during design stage. Classes and their relationships are represented in the Class models. Hence, Class models are natural inputs for the second test effort estimation process. Figure 2 shows an example Class model of “Retail Store Point of Sales” system [32].

3.2.1. Step 2.1: Class weight determination

For test effort estimation from Class models, all Classes belonging the OOS are required to be categorized according to their testing complexities [9,16]. The testing complexity of a Class is determined based on two types of information: (1) Information inherent to the Class and (2) Information about its interaction with other Classes. Information inherent to a Class are: (1) The number of attributes and (2) The number of methods. Information associated with its relationship with other Classes is (3) the number of services requested from other Classes. Based on the analysis done

in this work and in past studies, it was concluded that the testing complexity of a Class depends on above three parameters (Eq. (20)) [9,16,23].

$$Class_{weight} = f_{cw}(\#Attributes, \#Methods, \#Services_Requested) \quad (20)$$

Where, $\#Attributes$ and $\#Methods$ are the number of attributes and methods present inside the Class, $\#Services_Requested$ is the number of services requested from other Classes.

During literature survey, it was found that most of the studies have used only three complexity categories (Low/Simple, Average and High/Complex) to categorize the Classes [9,16]. From the experiments conducted in this works, it was concluded that using four categories (Simple, Medium, Complex and Very Complex) covers a wider range of projects resulting in better estimation accuracy [16]. Table 3 contains the complexity categories for the Classes. The weights assigned to the classes are well established in previously conducted research works and verified through the experiments [9,16,29].

After categorizing the user Classes to one of the four complexity classifications, they are assigned Class weights based on which of the four system component types (PDT, HIT, DMT and TMT) they belong to [9]. Table 4 contains Class weights assignments for all combinations of system component types and complexity classifications.

3.2.2. Step 2.2: Weight determination for Class models

The weight for each Class model is determined by combining the weights of all the Classes belonging to the model. The system may contain a number of

Table 4
Class weight assignments

System component type & description	Complexity			
	Simple	Medium	Complex	Very complex
PDT (Problem_Domain_Type)	3	6	10	15
HIT (Human_Interaction_Type)	4	7	12	19
DMT (Data_Management_Type)	5	8	13	20
TMT (Task_Management_Type)	4	6	9	13

Table 5
Technical and environmental factors with their weights

Factor ID	Factor description	Factor weight
1	Testing tools/test-ware (extent of testing automation)	3
2	Test basis (quality of information present in UML models)	5
3	Development environment (quality and suitability of development environment)	1
4	Test environment (quality and suitability of test environment/infrastructure)	1
5	Reusability (extent of component reusability)	2
6	Distributed system (extent of component distribution in multiple sites)	4
7	System performance requirements (system response/throughput requirements)	1
8	Security features (extent of security features required in the system)	2
9	Complexity of processing and interfacing	2

Class models. Therefore, the Total weight of the system is obtained by summing the Class model weights (Eq. (21)).

$$ClassModel_{weights} = \sum_{cm_1}^{cm_n} (Class_{weight}) \quad (21)$$

Where, $ClassModel_{weights}$ is the Total Weight of the system consisting of n Class models (cm_1 to cm_n), $\sum_{cm_1}^{cm_n} (Class_{weight})$ is the sum of Class Weights.

3.3. Step 3: Determination of adjustment factor and productivity factors

After estimating Total weight of the system based on Use Case and Class models, adjustments will be applied to it by taking into account the influences of Technical and Environmental Factors of the project/organization. During literature survey, the Technical and Environmental Factors pertinent to testing were collected and analyzed [6,7,9,16,19,26,29,30]. Based on analysis done, it was observed that some factors overlap while some others have minimal effect on test effort estimation [19]. In this study, a consolidated set of impactful factors were determined along with their degree of influence on testing in consultation with a number of test experts. The Delphi technique was used to get agreements on the factors. The results of the experiments have confirmed the validity of the selected factors. Table 5 contains the list of nine factors along with their weights (Factor_Weight) indicating their degree of importance with respect to testing. The weights are expressed in a scale of 1 to 5,

with 5 indicating highest and 1 indicating least degree of influence. In addition to the Factor_Weight, another value called Exhibit_Value was defined in this work. The Exhibit_Value represents the extent up to which the system exhibits each of the Adjustment Factors defined in Table 5. The Exhibit_Values are expressed in a scale of 1 to 5, with 5 indicating highest exhibit degree and 1 indicating lowest exhibit degree. These values are provided by the experts in the project team.

The Technical Environmental Factor TEF_{test} is calculated as the weighted sum based on formula given in Eq. (22). Then, the Adjustment Factor AF_{test} is calculated based on the well established formula given in Eq. (23) [7,30].

$$TEF_{test} = \sum_{i=1}^9 (Factor_Weight_i * Exhibit_Value_i) \quad (22)$$

$$AF_{test} = [0.65 + 0.01 * TEF_{test}] \quad (23)$$

The Adjusted Use Case model and Class model weights of the system are obtained by multiplying the Adjustment Factor AF_{test} to the Total weights of the system obtained from Eqs (19) and (21).

The final objective of getting Test Effort in Man-Hours is achieved by multiplying the Adjusted weights to the test team's Productivity Factor (PF). PF is defined by Man-Hours required to plan, write and conduct testing for one unit of Use Case/Class weight [30]. In this case two PFs ($Prodtest_ucm$ and $Prodtest_cm$) are required to estimate test efforts using the Use Case and the Class models.

Some studies have used a default PF irrespective of their project type and environment [16,30]. Others

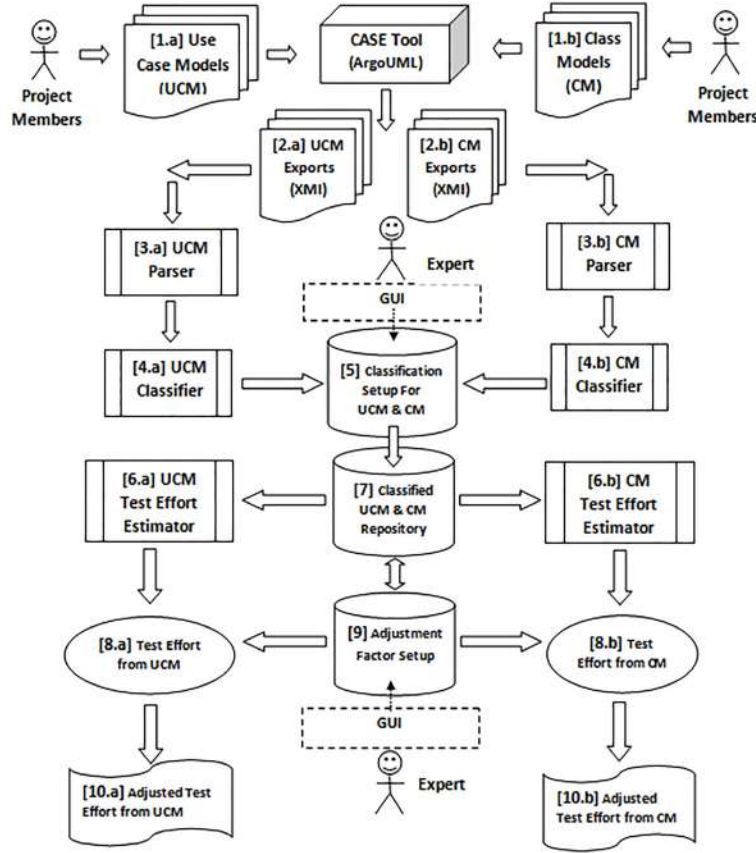


Fig. 3. System test effort estimator architecture.

have used three different PFs based on the project type and environment [11]. In papers [2,17], the authors have applied soft computing techniques to determine PF. As per the analysis done in this work, PF is highly dependent on the project type and environment [6]. Hence, this work recommends the use of organization's historical data of similar projects to determine the PF. This approach is justified by other studies [19] and experimentally verified in this work. In cases where historical data is unavailable or inadequate, opinion of test experts and experienced project members can be taken to determine PF.

3.4. Step 4: Final test effort estimation using Use Case and Class models

The final system test effort estimations using Use Case and Class models are represented by Eqs (24) and (25) respectively.

$$Test_Effort_{from_ucms} = UseCaseModel_{weights} * AF_{test} * Prod_{test_ucm} \quad (24)$$

Where, $UseCaseModel_{weights}$ is the total weight of the system derived from Use Case models, AF_{test} is the Adjustment Factor computed in Step-3 and $Prod_{test_ucm}$ is the test productivity factor corresponding to a single Use Case weight.

$$Test_Effort_{from_cms} = ClassModel_{weights} * AF_{test} * Prod_{test_cm} \quad (25)$$

Where, $ClassModel_{weights}$ is the total weight of the system derived from Class models, AF_{test} is the Adjustment Factor computed in Step-3 and $Prod_{test_cm}$ is the test productivity factor corresponding to a single Class weight.

The proposed test effort estimation approach enables the project team to get a better understanding of the testing complexity of the system along with required testing effort. The early test efforts obtained using Use Case models can be compared with test efforts obtained from Class models at a later stage.

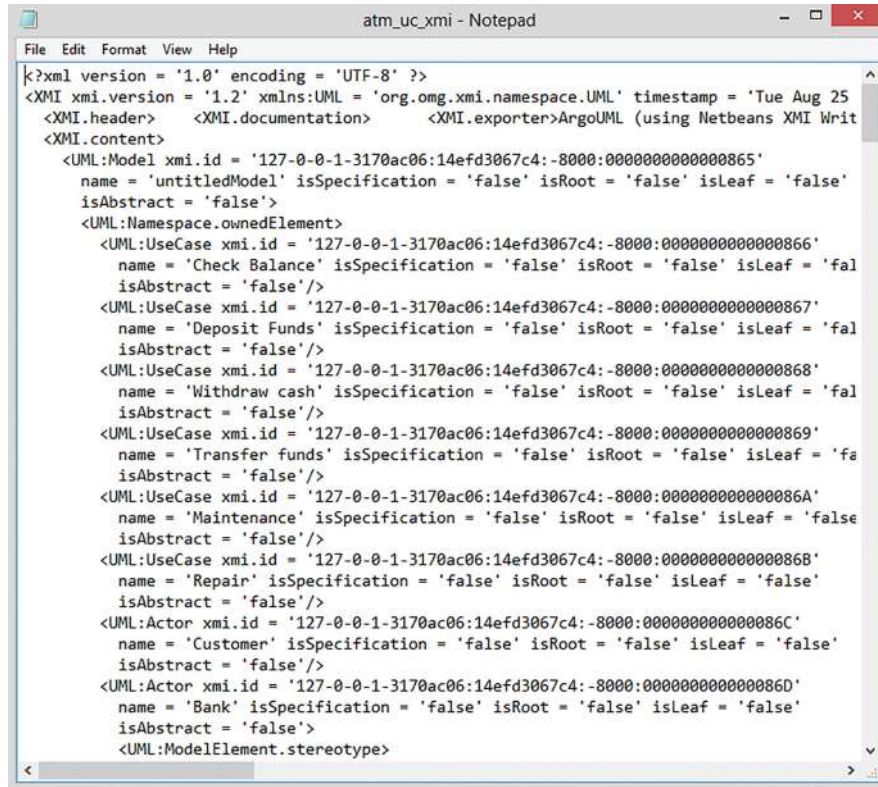


Fig. 4. Example XMI export of Use Case model.

4. Experimental studies and results analysis

In this section, the UML system test effort estimator architecture is presented along with the description of the implementation steps. The experimental studies are discussed next followed by the results analysis.

4.1. UML system test effort estimator architecture

The architecture of the proposed UML system test effort estimator with its major components is presented in Fig. 3. The main components of the estimator are: CASE tool ArgoUML, Use Case/Class model Parsers, Use Case/Class model Classifiers, Use Case/Class model Repository, Use Case/Class model test effort estimators and interfaces for classification/adjustment factor setup.

4.2. Implementation of UML system test effort estimators

For implementation of the estimators, a number of tools were used as described below. The UML models are created by organizations using CASE Tools that

support exporting of models to XMI (XML metadata interchange) format. For implementation purpose, ArgoUML CASE tool is used in this work. The parser programs are written using Java. MS SQL Server is used as the repository for all estimation related information. The interfaces are built using ASP.net, Angular JS and C-Sharp.

The implementation steps are presented below.

4.2.1. Step-1

The Use Case and Class models of the system created with required details (using CASE tool) are obtained during requirement and design stages. These models are then exported to XMI formats [18]. Additional information about Use Case scenarios are obtained from system's Use Case specifications. Figure 4 shows XMI export of Use Case model for ATM System.

4.2.2. Step-2

The 'Use Case model Parser' takes exported Use Case models as inputs and extracts required information about the Use Cases, Actors, their associations and stores in the model repository. Similarly,

Table 6
Summary of case study projects

Project ID	Project name	Application type/domain	Description	Technologies used
1	Remote GSM system	Remote GSM Switch Control Web Application	Control, monitoring and alarming of electronic gadgets through GSM	PHP, Html5, CSS3, Third party API as SMS Gateway, SQLserver
2	IDL security system	Web based security maintenance system	System to maintain authority and access level security to other systems	c#, ASP .Net, SQLserver
3	BIBM widget	BI tool for analysis of social networking data	Data analyze from social networking accounts	Java, Postgresql, Jdbc, Json, Ajax, Extjs 4.1.3, d3
4	KMS_Iret system	Web based application for procurement management	Facilitates procurement process including managing producer profiles, contracts, receiving, pricing and payments	J2SDK1.6, Oracle 10g
5	TicketBooking_PK	Web based Train Ticket Booking system	Train Ticket Booking, Reschedule and Cancellation	J2EE, HTML, CSS, Oracle 10g
6	Bakbd verification	Web Application for People Management	Employee Information Verification for authenticity	ASP.Net, C#.Net, SQLserver
7	Weather-TPI	Web based weather forecasting system	Weather forecasting application	ASP .Net, c#, Oracle10g
8	TRANS monitoring system	Web Application for Crime Management	Tracking, reporting and managing criminal complains	Java, J2EE, NetBeans, SQLserver
9	OMS for computer accessories	Web based Order Management system	Manage and process orders via company website	Visual Studio.Net-2010, Asp.Net with Ajax, VC#.Net, SQLserver
10	KV yojona	Web Application for Organic Farming and benefit distribution	Record block, subdivision, district-wise production and Prepare beneficiaries	ASP.NET using c#, ADO.NET, SQLserver
11	BMS staff welfare system	Web based application for corporate staff welfare	Facilitates employee interaction and welfare event conduction	J2SE, J2EE, HTML, CSS, SQLserver
12	Energy Utility – Billing Collection System	Web Application in utility domain	Billing and collection system to generate and pay electricity bill online	Java, Extjs 4.1, sql, HTML, CSS, Postgresql
13	Auto SMS responder	Android mobile app for responding to SMS	Auto responder app automatically responds to incoming SMS and missed call	App. Framework for Android, Dalvik VM, OpenGL ES 1.0, SQLite, Eclipse IDE
14	Healthcare-Adam	Web based healthcare application	Facilitates search and comparison of diseases	ASP.Net, Oracle10g
15	e-Counselling system	Web based admission for educational institutes	Facilitates admission process in various technical institutes	Html, Java Script, Asp.net, C#, ADO.NET, SQLserver
16	Cluster-based agricultural benefit allocation (CABA)-CBCD system	Web based application for agriculture sector	Facilitates agricultural benefit allocation at block, sub-division and district level	c#, ASP .Net, Oracle10g
17	CABA-CBTA system	Web based application for agriculture sector	Facilitates agency selection for supplying seeds and fertilizers	c#, ASP .Net, Oracle10g
18	CABA-CBTD system	Web based application for agriculture sector	Facilitates agricultural target benefit distribution	c#, ASP .Net, Oracle10g
19	CS LostFound system	Web based application for lost found items	Facilitates finding of lost items	Java, NetBeans, SQLserver

the ‘Class model Parser’ takes exported Class models as inputs. It extracts required information about the Classes, Attributes, Methods, Relationships and stores in the model repository.

4.2.3. Step-3

The ‘Use Case model classifier’ classifies the Use Cases and Actors based on the defined classification setup. Similarly, the ‘Class model classifier’ classifies the Classes based on the defined classification setup. The classification setups (as described in the ‘Proposed

Approach’ section) are maintained in the repository. A user interface is provided for creating/maintaining the classification setup information by the experts.

4.2.4. Step-4

Based on the outputs of Step-3, the ‘UCM and CM Test effort estimators’ compute Test efforts using Use Case and Class models of the system.

4.2.5. Step-5

The ‘Adjustment Factor’ is then computed based on the formulae specified in the ‘Proposed Approach’

Table 7
Estimated test efforts from Use Case and Class models vs Actual effort

Project ID	Estimated test effort from Use Case models [a]	Estimated test effort from Class models [b]	Actual test effort [c]	Variance in % for Use Case models $[((c-a)/c)*100]$	Variance in % for Class models $[((c-b)/c)*100]$
1	292.37	271.32	252	-16.02	-7.67
2	141.69	186.45	162	12.54	-15.09
3	120.39	153.9	138	12.76	-11.52
4	158.13	123.9	142	-11.36	12.75
5	300.58	312.3	380	20.9	17.81
6	319.44	265.63	298	-7.19	10.86
7	247.5	304.01	280	11.61	-8.57
8	292.65	189.72	237	-23.48	19.95
9	239.33	260.78	284	15.73	8.18
10	212.16	262.5	235	9.72	-11.7
11	166.73	152.06	138	-20.81	-10.19
12	127.89	148.34	140	8.65	-5.96
13	99.88	138.04	115	13.15	-20.03
14	237.23	177.05	205	-15.72	13.63
15	124.3	90.2	105	-18.38	14.09
16	89.73	157.33	128	29.9	-22.91
17	176.41	244.38	213	17.18	-14.73
18	84.86	155.55	122	30.44	-27.5
19	135.73	95.94	120	-13.11	20.05
Average effort variance				16.24	14.38

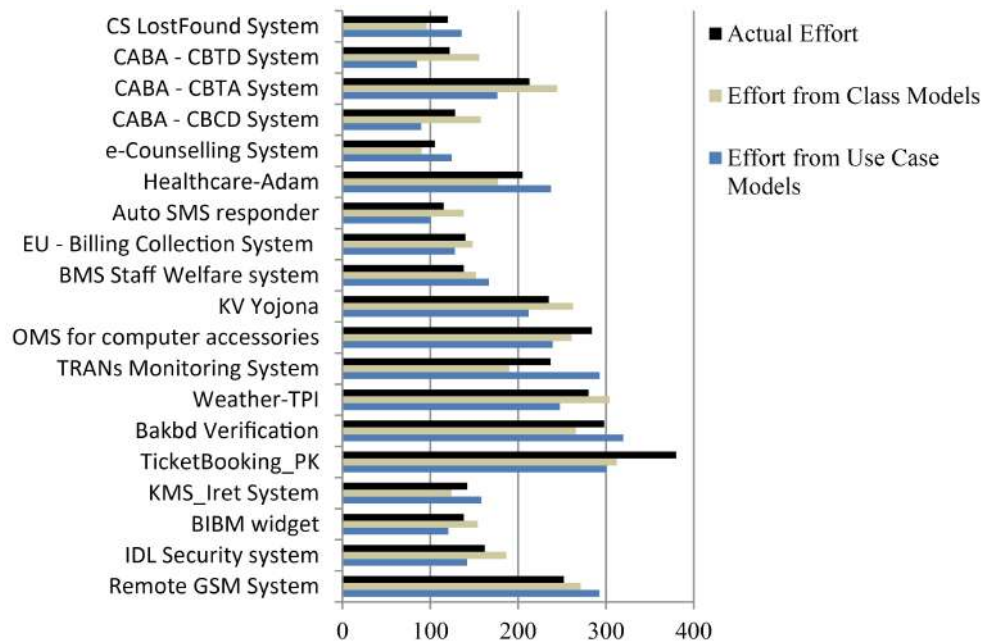


Fig. 5. Estimated test efforts from Use Case and Class models vs Actual effort.

section. A user interface is provided for maintaining the information about the Technical and Environmental Factors and their influences by the experts.

4.2.6. Step-6

Finally, the Adjusted Test efforts from Use Case and Class models are computed by multiplying the 'Adjustment Factor' and the corresponding 'Productivity Fac-

tors' (obtained from the historical data or from the experts in the project team through interviews) to the estimates obtained in Step-4.

4.3. Experimental studies

The experimental studies conducted in this work can be divided into three parts. (1) Selection of case study

projects and collection of data for System Test effort estimation. (2) Conduction of the experiments on case studies and (3) Analysis of results obtained.

4.3.1. Case studies

As part of the case studies, 19 recently completed web-based projects were selected from diversified domains and technologies [33]. Most of these projects were carried out in the IT departments of private and public sector organizations. Some of the projects are executed by final year master's degree students, under professional guidance, working as part of larger project teams in external organizations. Key members of the project teams were interviewed and the technical experts were consulted before selecting the projects. Table 6 contains the list of selected projects.

4.3.2. Experiments

After selection of case studies, the effort estimator environment is setup. This includes setting up the master data, Classification data, Adjustment Factors and their influence scores (for each case study) in the estimator repository. PFs were collected for each case study project. Once the environment is setup for all case studies, test effort estimation process is executed to generate estimates using Use Case and Class models [11].

4.3.3. Results analysis

In this section, the estimated test efforts for 19 case study projects are presented. Table 7 contains: (1) Estimated test efforts from both Use Case and Class models, (2) Actual test efforts incurred in the project and (3) Variances between them. The efforts are expressed in Man-Days.

Based on the results of the experiments (Table 7 and Fig. 5), the following inferences can be drawn. When comparing the Effort Variances for Use Case and Class models, it was observed that, in 13 out of 19 case studies, the estimations obtained from Class models exhibit a better accuracy than that from Use Case models. This observation is supported by the Average Effort Variance values. The average variance for Use Case model based estimation is 16.24 and that for the Class model based estimation is 14.38. This outcome supports the intuition that, the Class models developed in a later stage contain more information compared to the Use Case models which are developed earlier. The estimation accuracy of the proposed approach compares favorably to the accuracies observed by a number of similar studies [2,4,22,26]. However, the key contributing

factor for estimation accuracy is the level and precision of details available in the UML models.

The estimation accuracy of this work compares favorably to the accuracies observed by a number of similar studies. Nageswaran's [30] enhanced UCPM approach resulted in an estimation accuracy of 6%. However, his experiment was based on a single case study. Paper [2] obtained an estimation accuracy of 22% using soft computing techniques to enhance UCPM. Paper [22] achieved an estimation accuracy of 17% validated on a large incrementally developed project. Paper [9] obtained an aggregated MMRE of 0.18 using CP2 approach.

5. Conclusions and future work

Early estimation of test efforts is important for delivery of quality software products within strict timelines. As part of the literature survey, numerous existing estimation methods for both development and test effort estimation were studied along with their pros and cons. In spite of their obvious benefits, the main issues with the existing models are: lack of early estimations, inaccuracies, process complexities, single-stage approaches and time-consuming activities of gathering input metrics. The aim of this work is to assist project teams to plan early for the crucial system testing phase by providing system test effort estimations in early stages of SDLC. This work offers a simple, efficient, reasonably accurate, semi-automated approach for early system test effort estimation based on Use Case and Class models. As per the results obtained from experiments conducted with 19 case studies, it was concluded that Class models produce more accurate estimations than Use Case models. This observation is supported by the Average Effort Variance values of 16.24% and 14.38% corresponding to the Use Case and Class models. This is attributed to the fact that, Class models are developed in a later stage of SDLC when more information about the system is available. The estimation accuracy obtained by the proposed approach compares favorably to that of a number of similar studies. Having said that, the key contributing factor for estimation accuracy is the level and precision of details available in the UML models. The experiments were conducted on a number of mid-sized web applications. This work can be expanded to include more projects from diversified categories, sizes, technologies and domains. Further analysis of Adjustment Factors is required to study their influences on testing

of OOSs. In future, this estimation approach can be extended to produce further estimations using remaining UML models like Sequence, Activity and Statechart models.

References

- [1] A. Albrecht, Measuring application development productivity, in: *Proceedings of the Joint SHARE/GUIDE/IBM Application Development Symposium*, Vol 10, 1979, pp. 83–92.
- [2] A.B. Nassif, L.F. Capretz and D. Ho, Enhancing Use Case points estimation method using soft computing techniques, *Journal of Global Research in Computer Science* **1**(4) (2010), 12–21.
- [3] A. Hussain, A. Nadeem and M.T. Ikram, Review on formalizing Use Cases and scenarios: Scenario based testing, *International Conference on Emerging Technologies (ICET)*, IEEE, 2015, pp. 1–6.
- [4] B. Anda, H. Dreiem, D.I.K. Sjøberg and M. Jørgensen, Estimating software development effort based on Use Cases – Experiences from industry, *Springer, 2001-The Unified Modeling Language. Modeling Languages Concepts and Tools*, 2001, pp. 487–502.
- [5] B. Boehm, E. Horowitz, R. Madachy, D. Reifer, B. Clark, B. Steece, W. Brown, S. Chulani, and C. Abts, *Software Cost Estimation with COCOMO II*, Prentice Hall, 2000.
- [6] E.P.W.M. Van Veenendaal and T. Dekkers, Test point analysis: A method for test estimation, 1999.
- [7] E.R.C. de Almeida, B.T. de Abreu and R. Moraes, An alternative approach to test effort estimation based on use cases, *2009 International Conference on Software Testing Verification and Validation*, IEEE, 2009, pp. 279–288.
- [8] G. Booch, I. Jacobson and J. Rumbaugh, UML version 2.0, 2005.
- [9] G. Costagliola, F. Ferrucci, G. Tortora and G. Vitiello, Class point: An approach for the size estimation of object-oriented systems, *IEEE Transactions on Software Engineering* **31**(1) (2005), 52–74.
- [10] G. Karner, *Metrics for Objectory*, DiplomaThesis, University of Linköping, Sweden, 1993.
- [11] G. Schneider and J.P. Winters, *Applying Use Cases: A Practical Guide*, Pearson Education, 2001.
- [12] I. Jacobson, M. Christerson, P. Jonsson and G. Övergaard, *Object Oriented Software Engineering*, Addison Wesley, Harlow, 1992.
- [13] J. Capers, *Applied Software Measurement*, McGraw-Hill, 1996.
- [14] K. Kato, M. Sakawa and H. Katagiri, Interactive fuzzy programming based on expectation optimization and variance minimization for two-level stochastic linear programming problems, *KES Journal* **13** (2009), 111–118.
- [15] K.R. Jayakumar and A. Abran, A survey of software test estimation techniques, Scientific Research Publishing, *Journal of Software Engineering and Applications* **6**(10) (2016), 47–52.
- [16] K. SangEun, W. Lively, M. William and D.B. Simmons, An effort estimation by UML points in early stage of software development, *Software Engineering Research and Practice* (2006), 415–421.
- [17] M. Azzeh and A.B. Nassif, A hybrid model for estimating software project effort from Use Case Points, *Elsevier, Applied Soft Computing* **49** (2016), 981–989.
- [18] M. Dhineshkumar and Galeebathullah, An approach to generate test cases from sequence diagram, *IEEE, 2014 International Conference on Intelligent Computing Applications (ICICA)*, 2014, pp. 345–349.
- [19] M. Ochodek, J. Nawrocki and K. Kwarcia, Simplifying effort estimation based on Use Case Points, *Information and Software Technology* **53**(3) (2011), 200–213.
- [20] M. Sarma, and R. Mall, Automatic test case generation from UML models, *10th International Conference on. Information Technology, (ICIT 2007)*, IEEE, 2007, pp. 196–201.
- [21] P.C. Jorgensen, *Software Testing: A Craftsman's Approach*, CRC press, 2016.
- [22] P. Mohagheghi, B. Anda and R. Conradi, Effort estimation of use cases for incremental large-scale software development, *27th International Conference on Software Engineering, ACM*, 2005, pp. 303–311.
- [23] P. Sahoo and J.R. Mohanty, Early test effort prediction using UML Diagrams, *Indonesian Journal of Electrical Engineering and Computer Science* **5**(1) (2017), 220–228.
- [24] P. Sahoo and J.R. Mohanty, Early system test effort estimation automation for object-oriented systems, *Springer, Information and Decision Sciences*, 2018, pp. 325–333.
- [25] R. Anbunathan and A. Basu, Dataflow test case generation from UML Class diagrams, *IEEE International Conference on Computational Intelligence and Computing Research (IC-CIC)*, 2013, pp. 1–27.
- [26] R.K. Clemmons, Project estimation with Use Case Points, *The Journal of Defence Software Engineering*, 2006, 18–22.
- [27] R.V. Binder, *Testing Object-Oriented Systems: Models, Patterns, and Tools*, Addison-Wesley Professional, 2000.
- [28] S.K. Swain, D.P. Mohapatra and R. Mall, Test case generation based on state and activity models, *Journal of Object Technology* **9**(5) (2010), 1–27.
- [29] S.M. Satapathy, M. Kumar and S.K. Rath, Class point approach for software effort estimation using soft computing techniques, *2013 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, IEEE, 2013, pp. 178–183.
- [30] S. Nageswaran, Test effort estimation using use case points, *Quality Week* (2001), 1–6.
- [31] T.E. Ayyıldız and H.C. Terzi, Case study on software effort estimation, IACSIT press, *International Journal of Information and Electronics Engineering* **7**(3) (2017), 103–107.
- [32] <http://www.programsformca.com/search/label/UML%20Diagrams>.
- [33] <http://www.oss.models-db.com/>.